MINDANAO STATE UNIVERSITY
ILIGAN INSTITUTE OF TECHNOLOGY
COLLEGE OF COMPUTER STUDIES
DEPARTMENT OF COMPUTER APPLICATIONS

BCA151 Operating Systems Lab

Name	Reuyan, Alwyn Jade	Date	Oct 28, 2026
Course & Year	BSCA 3rd year		

Lab Activity 2

CPU Scheduling Algorithms

Introduction

CPU scheduling is an operating system function that determines which process in the ready queue receives CPU time. Different scheduling algorithms use different rules to determine execution order, affecting waiting time, turnaround time, response time, and overall system responsiveness.

In this laboratory activity, students will develop a CPU Scheduling Simulator using Python in GitHub Codespaces. They will implement and compare five algorithms: First-Come, First-Served (FCFS), non-preemptive Shortest Job First (SJF), non-preemptive Priority Scheduling, Round Robin (RR), and preemptive Shortest Remaining Time First (SRTF).

Learning Objectives

At the end of the activity, students should be able to:

- Create and manage a CPU scheduling laboratory repository using GitHub.
- Use GitHub Codespaces to develop and execute Python programs.
- Implement five CPU scheduling algorithms.
- Generate Gantt charts showing process execution.
- Calculate completion time, turnaround time, waiting time, and response time.
- Compare the performance of scheduling algorithms.
- Explain preemption, time quantum, starvation, and context-switching overhead.
- Commit and synchronize source code using GitHub.

Required Tools

- Existing GitHub account
- Web browser and internet connection
- GitHub Codespaces
- Python 3 environment

Dataset

Use the following dataset for FCFS, SJF, Priority Scheduling, and Round Robin.

Process	Arrival Time (AT)	Burst Time (BT)	Priority
P1	0	8	2
P2	0	4	1
P3	0	2	4
P4	0	6	3

Scheduling assumptions:

- A smaller priority number indicates a higher priority.
- Round Robin uses a time quantum of 2 ms.
- SJF and Priority Scheduling are non-preemptive.
- FCFS uses arrival order, with process ID breaking arrival-time ties.
- For SRTF, use the separate dataset below to demonstrate preemption meaningfully.

Separate SRTF dataset:

Process	Arrival Time (AT)	Burst Time (BT)	Priority
P1	0	8	2
P2	1	4	1
P3	2	2	4
P4	3	1	3

Step-by-Step Procedure Using GitHub

Part A. Create the GitHub Repository

1. Sign in to your existing account at https://github.com.
2. Click the + icon in the upper-right corner and select New repository.
3. Enter repository name: OS-Laboratory-2-CPU-Scheduling.
4. Enter description: Operating Systems Lab 2 - CPU Scheduling Algorithms.
5. Choose Private or Public.
6. Check Add a README file, then click Create repository.
7. In the repository, click Code, open the Codespaces tab, and select Create codespace on main.
8. Wait for the browser-based development environment to load.

Part B. Create the Python Source File

9. In Codespaces, locate the Explorer panel on the left.

10. Click New File and name it cpu_scheduling.py.
11. Open the file and enter the complete program below.
12. Save the file using Ctrl+S (Windows) or File > Save.

```
"""
Operating Systems Laboratory Activity 2
CPU Scheduling Simulator - Python 3

Implements five CPU scheduling algorithms:
    1. First-Come, First-Served (FCFS)
    2. Non-preemptive Shortest Job First (SJF)
    3. Non-preemptive Priority Scheduling
    4. Round Robin (RR)
    5. Preemptive Shortest Remaining Time First (SRTF)

For every algorithm the program prints a Gantt chart, a per-process
metrics table (CT, TAT, WT, RT) and the average metrics.
"""

from collections import deque
from copy import deepcopy

# ---------------------------------------------------------------------
# Dataset used by FCFS, non-preemptive SJF, Priority and Round Robin
# ---------------------------------------------------------------------
PROCESSES = [
    {"pid": "P1", "at": 0, "bt": 8, "priority": 2},
    {"pid": "P2", "at": 0, "bt": 4, "priority": 1},
    {"pid": "P3", "at": 0, "bt": 2, "priority": 4},
    {"pid": "P4", "at": 0, "bt": 6, "priority": 3},
]

# ---------------------------------------------------------------------
# Separate dataset for SRTF (different arrival times -> real preemption)
# ---------------------------------------------------------------------
SRTF_PROCESSES = [
    {"pid": "P1", "at": 0, "bt": 8, "priority": 2},
    {"pid": "P2", "at": 1, "bt": 4, "priority": 1},
    {"pid": "P3", "at": 2, "bt": 2, "priority": 4},
    {"pid": "P4", "at": 3, "bt": 1, "priority": 3},
]


def prepare(processes):
    """Return a deep copy with the bookkeeping fields reset."""
    result = deepcopy(processes)
    for p in result:
        p["remaining"] = p["bt"]
        p["first_start"] = None
        p["ct"] = None
    return result


def record_run(p, start, end, gantt):
    """Account for one CPU allocation of process p from start to end."""
    if start >= end:
        return
    if gantt and gantt[-1][2] == p["pid"] and gantt[-1][1] == start:
        gantt[-1] = (gantt[-1][0], end, p["pid"])
    else:
        gantt.append((start, end, p["pid"]))
    if p["first_start"] is None:
        p["first_start"] = start
```

```
    p["remaining"] -= end - start
    if p["remaining"] == 0:
        p["ct"] = end


def finish_results(processes):
    """Derive TAT, WT and RT once every process has completed."""
    for p in processes:
        p["tat"] = p["ct"] - p["at"]
        p["wt"] = p["tat"] - p["bt"]
        p["rt"] = p["first_start"] - p["at"]
    return processes


def fcfs(data):
    """First-Come, First-Served: arrival order, PID breaks ties."""
    ps = prepare(data)
    time, gantt = 0, []
    for p in sorted(ps, key=lambda x: (x["at"], x["pid"])):
        if time < p["at"]:
            time = p["at"]
        start = time
        time += p["bt"]
        record_run(p, start, time, gantt)
    finish_results(ps)
    return ps, gantt


def non_preemptive(data, mode):
    """Non-preemptive SJF (mode='sjf') or Priority (mode='priority')."""
    ps = prepare(data)
    time, done, gantt = 0, set(), []
    while len(done) < len(ps):
        ready = [p for p in ps if p["pid"] not in done and p["at"] <= time]
        if not ready:
            time = min(p["at"] for p in ps if p["pid"] not in done)
            continue
        if mode == "sjf":
            chosen = min(ready, key=lambda p: (p["bt"], p["at"], p["pid"]))
        else:
            chosen = min(ready, key=lambda p: (p["priority"], p["at"], p["pid"]))
        start = time
        time += chosen["bt"]
        record_run(chosen, start, time, gantt)
        done.add(chosen["pid"])
    finish_results(ps)
    return ps, gantt


def round_robin(data, quantum=2, stats=None):
    """Preemptive Round Robin; each ready process gets up to `quantum` ms.

    If a dict is passed as `stats`, it receives the number of CPU
    allocations (dispatches) performed by the scheduler.
    """
    if quantum <= 0:
        raise ValueError("Time quantum must be greater than zero.")
    ps = prepare(data)
    by_arrival = sorted(ps, key=lambda p: (p["at"], p["pid"]))
    ready, gantt = deque(), []
    time, i = 0, 0
    while i < len(by_arrival) or ready:
        if not ready:
            time = max(time, by_arrival[i]["at"])
        while i < len(by_arrival) and by_arrival[i]["at"] <= time:
```

```
            ready.append(by_arrival[i])
            i += 1
        p = ready.popleft()
        if stats is not None:
            stats["dispatches"] = stats.get("dispatches", 0) + 1
        start = time
        end = time + min(quantum, p["remaining"])
        record_run(p, start, end, gantt)
        time = end
        while i < len(by_arrival) and by_arrival[i]["at"] <= time:
            ready.append(by_arrival[i])
            i += 1
        if p["remaining"] > 0:
            ready.append(p)
    finish_results(ps)
    return ps, gantt


def srtf(data):
    """Preemptive Shortest Remaining Time First (event driven)."""
    ps = prepare(data)
    if not ps:
        return ps, []
    time = min(p["at"] for p in ps)
    completed, gantt = set(), []
    while len(completed) < len(ps):
        ready = [
            p for p in ps
            if p["at"] <= time and p["pid"] not in completed and p["remaining"] > 0
        ]
        if not ready:
            time = min(p["at"] for p in ps
                       if p["pid"] not in completed and p["at"] > time)
            continue
        current = min(ready, key=lambda p: (p["remaining"], p["at"], p["pid"]))
        upcoming = [p["at"] for p in ps
                    if p["pid"] not in completed and p["at"] > time]
        slice_end = time + current["remaining"]
        if upcoming and min(upcoming) < slice_end:
            slice_end = min(upcoming)
        record_run(current, time, slice_end, gantt)
        time = slice_end
        if current["remaining"] == 0:
            completed.add(current["pid"])
    finish_results(ps)
    return ps, gantt


def display(title, processes, gantt):
    """Print the Gantt chart, the metrics table and the averages."""
    print("\n" + "=" * 66)
    print(title)
    print("=" * 66)
    print("\nGANTT CHART")
    print(" | ".join(f"{pid} [{start}-{end}]" for start, end, pid in gantt))
    print("\nPROCESS METRICS")
    print(f"{'PID':<6}{'AT':>5}{'BT':>5}{'PR':>5}{'CT':>6}{'TAT':>6}{'WT':>6}{'RT':>6}")
    for p in sorted(processes, key=lambda x: x["pid"]):
        print(f"{p['pid']:<6}{p['at']:>5}{p['bt']:>5}{p['priority']:>5}"
              f"{p['ct']:>6}{p['tat']:>6}{p['wt']:>6}{p['rt']:>6}")
    n = len(processes)
    print(f"\nAverage Waiting Time: {sum(p['wt'] for p in processes) / n:.2f} ms")
    print(f"Average Turnaround Time: {sum(p['tat'] for p in processes) / n:.2f} ms")
    print(f"Average Response Time: {sum(p['rt'] for p in processes) / n:.2f} ms")
```

```
def main():
    ps, g = fcfs(PROCESSES)
    display("1. FCFS", ps, g)

    ps, g = non_preemptive(PROCESSES, "sjf")
    display("2. NON-PREEMPTIVE SJF", ps, g)

    ps, g = non_preemptive(PROCESSES, "priority")
    display("3. NON-PREEMPTIVE PRIORITY", ps, g)

    ps, g = round_robin(PROCESSES, quantum=2)
    display("4. ROUND ROBIN (QUANTUM = 2)", ps, g)

    ps, g = srtf(SRTF_PROCESSES)
    display("5. SRTF (SEPARATE ARRIVAL-TIME DATASET)", ps, g)


if __name__ == "__main__":
    main()
```

Part C. Run the Program

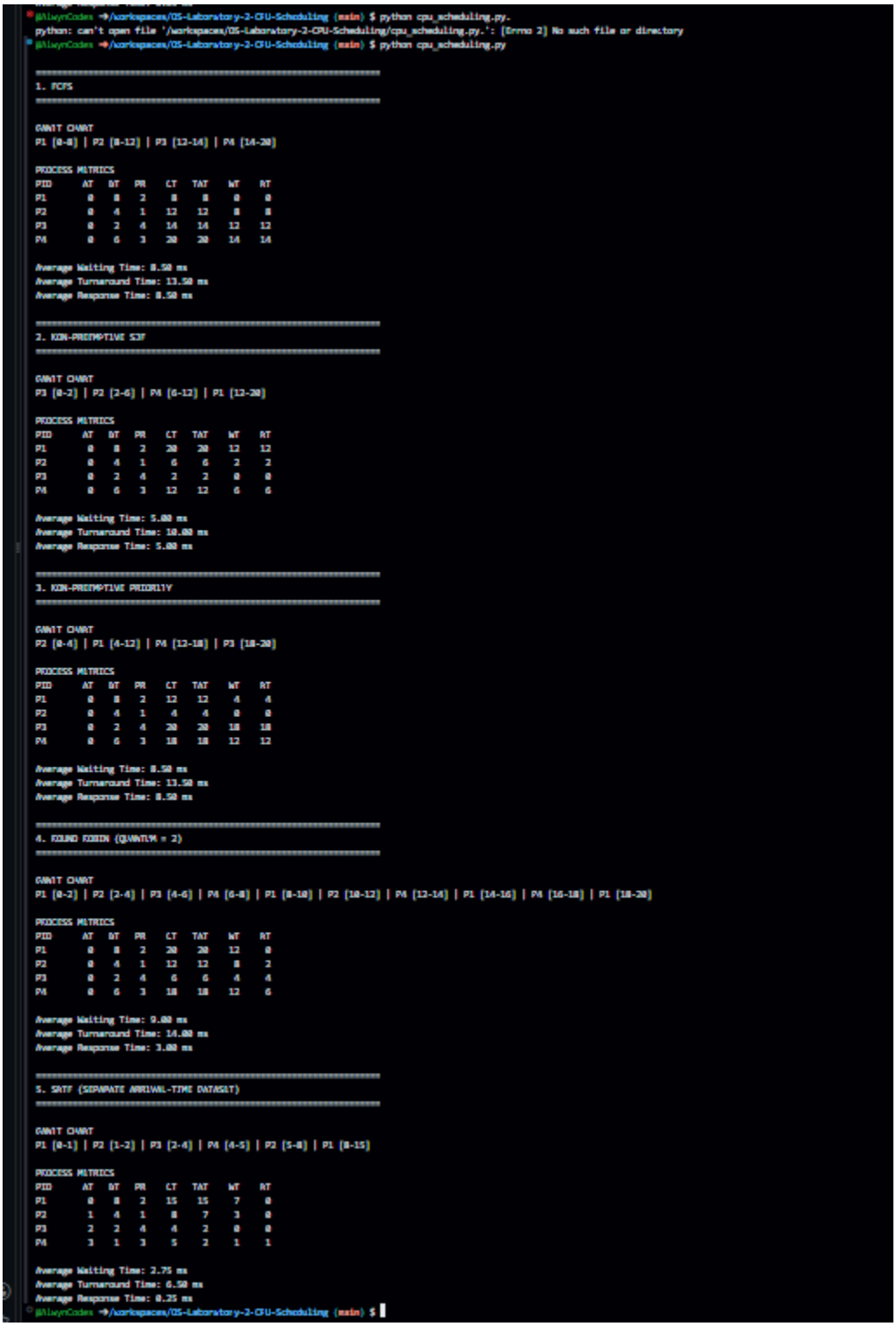
13. In Codespaces, select Terminal > New Terminal.
14. Run python3 cpu_scheduling.py. If unavailable, try python cpu_scheduling.py.
15. Examine the Gantt chart, process metrics, and averages for each algorithm.
16. Take a screenshot of the terminal output for your report.

Reference Results

For the first four algorithms, the average metrics should match these values:

Algorithm	Average WT	Average TAT	Average RT
FCFS	8.50 ms	13.50 ms	8.50 ms
Non-preemptive SJF	5.00 ms	10.00 ms	5.00 ms
Non-preemptive Priority	8.50 ms	13.50 ms	8.50 ms
Round Robin (q = 2)	9.00 ms	14.00 ms	3.00 ms

Actual Results Screenshot:



SRTF uses the separate dataset because its processes have different arrival times.

Part D. Verify the Results Manually

17. Draw a Gantt chart for each algorithm and check the execution order.
18. Verify these orders for the original dataset: FCFS: P1 → P2 → P3 → P4; SJF: P3 → P2 → P4 → P1; Priority: P2 → P1 → P4 → P3.
19. For Round Robin, verify that each ready process receives up to 2 ms per turn and unfinished processes return to the end of the ready queue.
20. For SRTF, identify where the running process is preempted when another process has a shorter remaining burst time.
21. Manually calculate at least two processes for every algorithm and compare your answers with the program.

Use these formulas:

Metric	Formula
Turnaround Time (TAT)	TAT = CT − AT
Waiting Time (WT)	WT = TAT − BT
Response Time (RT)	RT = First CPU Start − AT
Average metric	Sum of metric values ÷ number of processes

Example: In FCFS, P2 starts at 8 ms and finishes at 12 ms. Thus CT = 12, TAT = 12 − 0 = 12 ms, WT = 12 − 4 = 8 ms, and RT = 8 − 0 = 8 ms.

Part E. Experiment with Different Parameters

Step 22. Change the Round Robin Time Quantum

Find this line:

```
ps, g = round_robin(PROCESSES, quantum=2)
```

Change the quantum to 4:

```
ps, g = round_robin(PROCESSES, quantum=4)
```

Save and rerun the program. Then change the quantum to 1 and run it again. Record the average waiting time, turnaround time, and response time.

The program was rerun with q = 1, 2, 3, 4 and 8 on the shared dataset. The table records the averages printed by the program and the number of CPU allocations (dispatches) it performed; each allocation is a potential context switch.

Table 1. *Round Robin time-quantum experiment (shared dataset).*

Quantum (ms)	CPU allocations	Avg WT (ms)	Avg TAT (ms)	Avg RT (ms)
1	20	9.50	14.50	1.50
2	10	9.00	14.00	3.00
3	8	10.25	15.25	4.25
4	6	9.00	14.00	5.50
8	4	8.50	13.50	8.50

- How does the time quantum affect the number of CPU allocations?
- Which quantum produces the lowest average waiting time in your tests?
- What happens when the quantum becomes very large?
- Why can a very small quantum increase context-switching overhead in a real operating system?

Findings on the Time Quantum

1. The number of CPU allocations falls sharply as the quantum grows: 20 at q = 1, 10 at q = 2, 8 at q = 3, 6 at q = 4 and only 4 at q = 8. Every allocation is a potential context switch, so a small quantum multiplies scheduling and switching overhead.
2. q = 2 and q = 4 both give the lowest average waiting time, 9.00 ms. q = 1 is worse (9.50 ms) because the larger number of turns works against the long processes, and q = 3 is worse still (10.25 ms) because it leaves awkward remainders against bursts of 2, 4, 6 and 8 ms. The smallest quantum is therefore not automatically the best.
3. When the quantum reaches the longest burst (q = 8) every process finishes in its first turn and Round Robin degenerates into FCFS: the averages become 8.50 / 13.50 / 8.50 ms, exactly the FCFS figures, and average response time rises to 8.50 ms.
4. A very small quantum makes the context-switch cost comparable to the quantum itself. The CPU then spends a large share of its time saving and restoring process state instead of executing user code, so throughput falls even though response time looks good. Real systems keep the quantum well above the switch cost and often size it so that about 80% of CPU bursts are shorter than one quantum.

Step 23. Modify the Process Dataset

Change burst times and priorities, then rerun the program. Use the same modified dataset for FCFS, SJF, Priority Scheduling, and Round Robin to compare them fairly. Keep the SRTF dataset separate unless you deliberately change its arrival times.

Table 2. *Modified dataset used in Step 23 (arrival times kept at 0 for a fair comparison).*

Process	Arrival Time (AT)	Burst Time (BT)	Priority

P1	0	5	3
P2	0	9	1
P3	0	3	4
P4	0	7	2

The SRTF row keeps its own arrival times (0, 1, 2 and 3 ms) but uses the same new burst times and priorities, so that preemption can still be observed.

Table 3. *Average metrics obtained with the modified dataset.*

Algorithm	Average WT	Average TAT	Average RT
FCFS	9.00 ms	15.00 ms	9.00 ms
SJF	6.50 ms	12.50 ms	6.50 ms
Priority	11.50 ms	17.50 ms	11.50 ms
Round Robin	13.00 ms	19.00 ms	3.00 ms
SRTF	5.50 ms	11.50 ms	5.50 ms

With the modified dataset SJF still gives the lowest average waiting time of the four shared-dataset algorithms (6.50 ms). Priority Scheduling becomes the worst (11.50 ms) because the longest job, P2 (9 ms), now also has the highest priority and blocks the other three for 9 ms — the opposite of the shared dataset, where the longest job had low priority. Round Robin's average waiting time rises to 13.00 ms while its average response time stays at 3.00 ms, and SRTF remains the best overall on its own arrival-time dataset.

Part F. Save Your Work to GitHub

22. Click Source Control in the left sidebar.
23. Review the changed file cpu_scheduling.py.
24. Enter commit message: Implement CPU scheduling algorithms.
25. Click Commit and confirm if prompted.
26. Click Sync Changes or the available push option to upload your commit to GitHub.
27. Open your repository on GitHub and verify that the latest cpu_scheduling.py file is present.

Repository Link : https://github.com/AlwynCodes/OS-Laboratory-2-CPU-Scheduling

Results and Analysis

The five algorithms were run on the datasets supplied in this activity using cpu_scheduling.py. Every value in this section was printed by the program and then checked by hand using the formulas in Part D: TAT = CT − AT, WT = TAT − BT and RT = first CPU start − AT.

Gantt Charts and Calculation Tables

Each bar in a Gantt chart is one CPU allocation and the numbers are times in milliseconds. FCFS, SJF, Priority and Round Robin use the shared dataset (all arrival times 0); SRTF uses its separate arrival-time dataset.

1. First-Come, First-Served (FCFS)



Figure 1. *FCFS — execution order P1 → P2 → P3 → P4.*

Table 4. *Process metrics for FCFS (times in ms).*

PID	AT	BT	PR	CT	TAT	WT	RT
P1	0	8	2	8	8	0	0
P2	0	4	1	12	12	8	8
P3	0	2	4	14	14	12	12
P4	0	6	3	20	20	14	14
Average					**13.50**	**8.50**	**8.50**

All four processes arrive at time 0, so FCFS runs them in PID order. P1's 8 ms burst delays every later process — the convoy effect — and P3 ends up waiting 12 ms and P4 14 ms.

2. Non-Preemptive Shortest Job First (SJF)



Figure 2. *SJF — execution order P3 → P2 → P4 → P1.*

Table 5. *Process metrics for SJF (times in ms).*

PID	AT	BT	PR	CT	TAT	WT	RT
P1	0	8	2	20	20	12	12
P2	0	4	1	6	6	2	2
P3	0	2	4	2	2	0	0

P4	0	6	3	12	12	6	6
Average					**10.00**	**5.00**	**5.00**

The shortest job, P3 (2 ms), runs first and the longest, P1 (8 ms), runs last. Removing the convoy effect cuts average waiting time from 8.50 ms to 5.00 ms.

3. Non-Preemptive Priority Scheduling



Figure 3. *Priority — execution order P2 → P1 → P4 → P3 (1 is the highest priority).*

Table 6. *Process metrics for Priority (times in ms).*

PID	AT	BT	PR	CT	TAT	WT	RT
P1	0	8	2	12	12	4	4
P2	0	4	1	4	4	0	0
P3	0	2	4	20	20	18	18
P4	0	6	3	18	18	12	12
Average					**13.50**	**8.50**	**8.50**

P2 has the highest priority (1) and runs first; P3 has the lowest (4) and waits 18 ms. The averages coincide with FCFS, but the waiting time is spread far less evenly — the pattern behind starvation.

4. Round Robin (q = 2 ms)

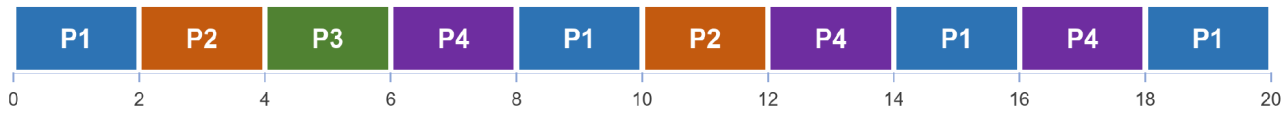


Figure 4. *Round Robin with q = 2 — ten CPU allocations.*

Table 7. *Process metrics for Round Robin (times in ms).*

PID	AT	BT	PR	CT	TAT	WT	RT
P1	0	8	2	20	20	12	0
P2	0	4	1	12	12	8	2
P3	0	2	4	6	6	4	4
P4	0	6	3	18	18	12	6
Average					**14.00**	**9.00**	**3.00**

Ten allocations cover the 20 ms of work. P3 finishes first at 6 ms, while the long jobs P1 and P4 need three turns each. Response time improves to 3.00 ms, but the extra turns raise average waiting time to 9.00 ms.

5. Preemptive Shortest Remaining Time First (SRTF)

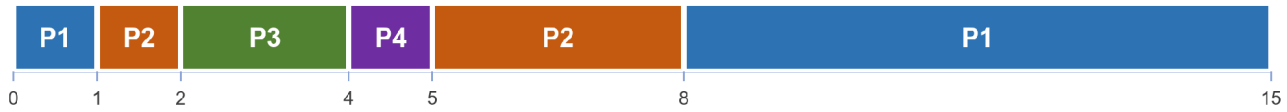


Figure 5. *SRTF on the separate arrival-time dataset — P1 is preempted at 1, 2 and 3 ms.*

Table 8. *Process metrics for SRTF (times in ms).*

PID	AT	BT	PR	CT	TAT	WT	RT
P1	0	8	2	15	15	7	0
P2	1	4	1	8	7	3	0
P3	2	2	4	4	2	0	0
P4	3	1	3	5	2	1	1
Average					**6.50**	**2.75**	**0.25**

P1 starts first but is preempted three times as shorter jobs arrive, resumes only at 8 ms and finishes at 15 ms. Preemption produces the best averages in the activity: WT 2.75 ms, TAT 6.50 ms and RT 0.25 ms.

Comparison of Average Metrics

Table 9. *Average metrics for all five algorithms.*

Algorithm	Dataset	Avg WT (ms)	Avg TAT (ms)	Avg RT (ms)
FCFS	Shared	8.50	13.50	8.50
SJF (non-preemptive)	Shared	5.00	10.00	5.00
Priority (non-preemptive)	Shared	8.50	13.50	8.50
Round Robin (q = 2)	Shared	9.00	14.00	3.00
SRTF (preemptive)	Separate	2.75	6.50	0.25

SRTF is not directly comparable with the other four: it runs on a dataset with different arrival times, which is why the two groups are shown separately.

What the Comparison Shows

- SJF has the lowest average waiting time on the shared dataset (5.00 ms) — 3.50 ms below FCFS and Priority and 4.00 ms below Round Robin — and the lowest average turnaround time as well (10.00 ms).
- Priority Scheduling matches the FCFS averages exactly (8.50 / 13.50 / 8.50 ms). This is a coincidence of the dataset: ordering the bursts 4, 8, 6, 2 (Priority) and 8, 4, 2, 6 (FCFS) happens to give the same total waiting time of 34 ms. The per-process values differ sharply — P3 waits 18 ms under Priority but 12 ms under FCFS.
- Round Robin has the highest average waiting time of the four (9.00 ms) but the lowest average response time (3.00 ms): every process starts within 6 ms of arrival, whereas under FCFS the last process waits 14 ms before its first allocation. That is precisely the trade-off it is designed for.

- SRTF produces the lowest averages in the whole activity (2.75 / 6.50 / 0.25 ms) by preempting P1 three times, but it performs six CPU allocations for four processes instead of four — the extra cost of preemption.
- Waiting-time spread matters as much as the average. FCFS ranges from 0 to 14 ms, Priority from 0 to 18 ms, SJF from 0 to 12 ms, and Round Robin only from 4 to 12 ms, which makes Round Robin the most predictable of the four.

Questions

1. What is CPU scheduling, and why is it necessary?
2. What is the difference between FCFS and SJF?
3. Why can SJF produce a lower average waiting time than FCFS?
4. How does Priority Scheduling determine the next process?
5. What is starvation? Which algorithms can experience it?
6. How does Round Robin improve responsiveness for interactive processes?
7. What is the effect of changing the Round Robin time quantum?
8. How does SRTF differ from non-preemptive SJF?
9. Why do preemptive algorithms require the operating system to save and restore process context?
10. Does the algorithm with the lowest average waiting time always provide the best user experience? Explain.
11. Why is it important to use the same dataset when comparing algorithms?
12. What role does GitHub play in managing and submitting your laboratory work?

Answers to the Questions

1. CPU scheduling is the operating-system function that decides which process in the ready queue receives the CPU next, and for how long. It is necessary because a single CPU cannot execute every ready process at once: the scheduling decision determines execution order and therefore waiting time, turnaround time, response time, throughput and fairness. Without it the system could not share the CPU among competing processes, keep the processor busy while other processes wait for I/O, or give interactive work timely service.

2. FCFS is non-preemptive and runs processes strictly in arrival order, with the process ID breaking arrival-time ties: the first process to arrive runs to completion no matter how long it is. SJF is also non-preemptive here, but it selects the ready process with the smallest burst time. FCFS is simple and cannot starve a process, but it suffers from the convoy effect; SJF minimises average waiting time, but it needs burst-time estimates and can starve long jobs.

3. SJF schedules the shortest jobs first, so a short job never waits behind a long one. A process's waiting time is the sum of the bursts of every process scheduled before it, and sorting those bursts from smallest to largest minimises that sum — hence the average. On this dataset SJF gives 5.00 ms against 8.50 ms for FCFS: P1 (8 ms) is moved to the end and the three short jobs no longer queue behind it.

4. Priority Scheduling selects the ready process with the highest priority, which in this activity means the smallest priority number. With the shared dataset the order is P2 (priority 1), P1 (2), P4 (3) and P3 (4); ties would be broken by arrival time and then by process ID. The version implemented here is non-preemptive,

so a running process is never interrupted when a higher-priority process arrives; a preemptive variant would switch immediately.

5. Starvation is the indefinite postponement of a process because other processes are always selected ahead of it. SJF and SRTF can starve long jobs when short jobs keep arriving, and Priority Scheduling can starve low-priority processes: here P3 waits 18 ms while P2 waits 0 ms, and a steady stream of higher-priority arrivals could postpone P3 forever. FCFS and Round Robin do not starve processes in normal operation. Aging — gradually raising the priority of waiting processes — is the standard remedy.

6. Round Robin gives every ready process a turn of at most one quantum, so no process waits longer than roughly $(n - 1) \times q$ before its first CPU allocation. That bound makes response time predictable, which is what interactive processes need: a keystroke or a click is serviced within a few quanta instead of waiting for a long batch job to finish. On the shared dataset Round Robin's average response time (3.00 ms) is the lowest of the four algorithms.

7. The quantum trades responsiveness against overhead. A small quantum shortens the wait before a process first runs but multiplies the number of dispatches and context switches; a large quantum reduces switches but makes Round Robin behave like FCFS, so interactive processes wait longer. The measurements show both effects: allocations fall from 20 ($q = 1$) to 4 ($q = 8$), average response time rises from 1.50 ms to 8.50 ms, and at $q = 8$ the averages are identical to FCFS. In practice the quantum is kept much larger than the context-switch time, often so that about 80% of CPU bursts are shorter than one quantum.

8. SRTF is the preemptive version of SJF: instead of choosing the shortest job only when the CPU becomes free, it compares the remaining burst time of the running process with that of every newly arrived process and preempts whenever a shorter one appears. On the SRTF dataset P1 is preempted at 1, 2 and 3 ms before it finally resumes at 8 ms. SRTF therefore reacts faster to short jobs and usually improves average waiting and response time, at the cost of more context switches and a higher risk of starving long jobs.

9. A preempted process is not finished — it must later resume exactly where it stopped. The operating system therefore saves its context (program counter, CPU registers, stack pointer, status flags and memory-management information) in its process control block, and restores the context of the incoming process. Without this save and restore, the interrupted process would resume with the wrong program counter or register values and would produce incorrect results or crash.

10. No. Average waiting time is only one dimension of performance. SJF has the lowest average waiting time here (5.00 ms) but starves long jobs and needs burst times that a real system does not know in advance. FCFS is simple and fair in arrival order but causes the convoy effect and poor response time. Round Robin has the highest average waiting time of the four (9.00 ms) yet by far the best response time (3.00 ms), which is what a user actually perceives. Fairness, predictability and the spread of waiting times matter as much as the average, so the best algorithm depends on the workload.

11. The algorithms must be compared on identical input; otherwise differences in arrival times, burst times or priorities are mixed with the differences caused by the algorithm, and the comparison is meaningless. Holding the dataset constant keeps every variable except the scheduling rule fixed — which is exactly why SRTF is given its own dataset with different arrival times and is not compared line-for-line with the other four.

12. GitHub supplies the version control, hosting and cloud development environment for the activity. The repository stores cpu_scheduling.py together with a history of every commit, GitHub Codespaces provides the Python 3 environment used to write and run the program without a local installation, and committing and syncing uploads the work so that the instructor can verify it. The commit history also documents how the code evolved and makes the submission reproducible and shareable.

Conclusions

This activity implemented five CPU scheduling algorithms in Python — FCFS, non-preemptive SJF, non-preemptive Priority, Round Robin and preemptive SRTF — and used them to schedule the datasets supplied in the worksheet. The simulator produced a Gantt chart and a full set of metrics for every algorithm, and the four shared-dataset averages matched the reference values exactly (FCFS 8.50 / 13.50 / 8.50 ms, SJF 5.00 / 10.00 / 5.00 ms, Priority 8.50 / 13.50 / 8.50 ms and Round Robin 9.00 / 14.00 / 3.00 ms), confirming that the implementation and the manual calculations agree.

The results show that no single algorithm is best on every measure. SJF gave the lowest average waiting and turnaround times on the shared dataset because it runs the short jobs first and removes the convoy effect that penalises FCFS. Priority Scheduling produced averages identical to FCFS here, but its per-process values were far less even — P3 waited 18 ms — which is precisely the pattern that leads to starvation when low-priority work competes with a steady stream of high-priority arrivals. Round Robin accepted the highest average waiting time (9.00 ms) in exchange for the lowest response time (3.00 ms), the property that makes it suitable for interactive systems. SRTF, on its own arrival-time dataset, achieved the best figures in the activity (WT 2.75 ms, TAT 6.50 ms, RT 0.25 ms) by preempting P1 three times in favour of shorter jobs, at the cost of more context switches and a greater risk of starving long processes.

The time-quantum experiment showed that the quantum is a tuning parameter rather than a constant. Decreasing it from 8 to 1 increased the number of CPU allocations from four to twenty and reduced average response time from 8.50 ms to 1.50 ms, while average waiting time moved non-monotonically (9.50, 9.00, 10.25, 9.00 and 8.50 ms for q = 1, 2, 3, 4 and 8). At q = 8 the quantum exceeded every burst and Round Robin degenerated into FCFS; at q = 1 the switching overhead would dominate in a real system. The modified-dataset experiment confirmed the same ranking — SJF best (6.50 ms) and Priority worst (11.50 ms) once the longest job also had the highest priority — which supports the conclusion that the ranking depends on the workload as much as on the algorithm.

Overall, the activity demonstrated the central trade-offs of CPU scheduling: throughput and average waiting time against response time, and fairness and simplicity against optimality. It also showed why the operating system must save and restore process context on every preemption, and why burst times must be estimated rather than known in practice — a limitation of this simulator, which assumes they are given. Extending it with I/O bursts, an explicit context-switch cost and a multilevel feedback queue would bring it closer to a real scheduler. All source code was committed and synchronised through GitHub, so the work is versioned, reproducible and ready for submission.

Required Deliverables

- GitHub repository link
- Completed Python source code
- Screenshot of the program running in Codespaces
- Gantt charts for all five algorithms
- Calculation tables showing CT, TAT, WT, and RT
- Comparison table of average metrics
- Results of the Round Robin time-quantum experiment
- Answers to the laboratory questions
- Brief analysis and conclusion

Appendix A. Program Output

Console output of python cpu_scheduling.py, produced in the Codespaces terminal. It contains the Gantt chart, metrics table and averages for each of the five algorithms.

```
======================================================================
1. FCFS
======================================================================

GANTT CHART
P1 [0-8] | P2 [8-12] | P3 [12-14] | P4 [14-20]

PROCESS METRICS
PID      AT   BT   PR    CT   TAT    WT    RT
P1        0    8    2     8     8     0     0
P2        0    4    1    12    12     8     8
P3        0    2    4    14    14    12    12
P4        0    6    3    20    20    14    14

Average Waiting Time: 8.50 ms
Average Turnaround Time: 13.50 ms
Average Response Time: 8.50 ms

======================================================================
2. NON-PREEMPTIVE SJF
======================================================================

GANTT CHART
P3 [0-2] | P2 [2-6] | P4 [6-12] | P1 [12-20]

PROCESS METRICS
PID      AT   BT   PR    CT   TAT    WT    RT
P1        0    8    2    20    20    12    12
P2        0    4    1     6     6     2     2
P3        0    2    4     2     2     0     0
P4        0    6    3    12    12     6     6

Average Waiting Time: 5.00 ms
Average Turnaround Time: 10.00 ms
Average Response Time: 5.00 ms

======================================================================
3. NON-PREEMPTIVE PRIORITY
======================================================================

GANTT CHART
P2 [0-4] | P1 [4-12] | P4 [12-18] | P3 [18-20]

PROCESS METRICS
PID      AT   BT   PR    CT   TAT    WT    RT
P1        0    8    2    12    12     4     4
P2        0    4    1     4     4     0     0
P3        0    2    4    20    20    18    18
P4        0    6    3    18    18    12    12

Average Waiting Time: 8.50 ms
Average Turnaround Time: 13.50 ms
Average Response Time: 8.50 ms

======================================================================
4. ROUND ROBIN (QUANTUM = 2)
======================================================================
```

```
GANTT CHART
P1 [0-2] | P2 [2-4] | P3 [4-6] | P4 [6-8] | P1 [8-10] | P2 [10-12] | P4 [12-14] | P1 [14-16] | P4 [16-18] | P1
[18-20]

PROCESS METRICS
PID      AT   BT   PR    CT   TAT    WT    RT
P1        0    8    2    20    20    12     0
P2        0    4    1    12    12     8     2
P3        0    2    4     6     6     4     4
P4        0    6    3    18    18    12     6

Average Waiting Time: 9.00 ms
Average Turnaround Time: 14.00 ms
Average Response Time: 3.00 ms

================================================================
5. SRTF (SEPARATE ARRIVAL-TIME DATASET)
================================================================

GANTT CHART
P1 [0-1] | P2 [1-2] | P3 [2-4] | P4 [4-5] | P2 [5-8] | P1 [8-15]

PROCESS METRICS
PID      AT   BT   PR    CT   TAT    WT    RT
P1        0    8    2    15    15     7     0
P2        1    4    1     8     7     3     0
P3        2    2    4     4     2     0     0
P4        3    1    3     5     2     1     1

Average Waiting Time: 2.75 ms
Average Turnaround Time: 6.50 ms
Average Response Time: 0.25 ms
```